

PROJECT REPORT

Automated Threat Detection Pipeline

Infrastructure: Distributed Systems Lab (Raspberry Pi & Proxmox PVE)

Author: Jacek Kaplon | Date: June 16, 2026

1. Executive Summary

This engineering document details the implementation of an automated threat detection and enrichment pipeline. The architecture integrates a containerized, low-interaction SSH/Telnet honeypot (Cowrie) deployed on a Raspberry Pi node with a centralized Wazuh SIEM platform hosted in a Proxmox VE environment. The system automates network threat ingestion, real-time File Integrity Monitoring (FIM), cryptographic hash generation, and instant multi-site security correlation using the VirusTotal REST API. The ultimate objective is to eliminate manual triage overhead for dropped malware payloads.

2. Technical Stack & Infrastructure Layer

The infrastructure is engineered in a decoupled multi-tier topology to ensure host isolation and security telemetry integrity:

- **Honeypot Sensor:** Cowrie deployed via Docker containers on a Raspberry Pi node. It emulates standard SSH environments, capturing brute-force credentials and secondary binary payloads.
- **Endpoint Agent:** Wazuh Agent deployed on the Debian 13 host layer. It actively monitors target container storage blocks.
- **SIEM Engine:** Centralized Wazuh Manager instance running inside a Proxmox VE container, performing alert aggregation, log parsing, and correlation.
- **Threat Intelligence:** VirusTotal Public REST API integration used for checking cryptographic signatures against active multi-vendor detection repositories.

3. Technical Implementation & Persistence Mapping

To prevent the loss of volatile threat data during Docker container lifecycles, a persistent volume bind mount is configured. Captured malware binaries are mapped from the container layer directly onto the host storage block under the following deployment path:

```
/var/lib/docker/volumes/<volume_id>/_data/lib/cowrie/downloads
```

3.1. File Integrity Monitoring (FIM) Engine

The local Wazuh Agent monitors the persistence path in real-time. The active ossec.conf block utilises the inotify engine to watch for filesystem write changes, computing cryptographic hashes immediately upon file closure:

```
<syscheck>
  <directories realtime="yes" check_sha256="yes">
    /var/lib/docker/volumes/<volume_id>/_data/lib/cowrie/downloads
  </directories>
</syscheck>
```

4. System Verification & Incident Simulation

To validate the operational readiness of the end-to-end detection pipeline, a live exploitation lifecycle was simulated on the production node using a non-destructive malware signature.

Step 1: Dropping the Controlled Malware Payload (EICAR)

The standardised EICAR test string was introduced directly into the monitored Cowrie downloads path. To prevent the local shell from misinterpreting special character markers, single-quote encapsulation was enforced:

```
echo 'X50!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' > test_eicar.txt
```

Step 2: Local Agent Ingestion Check

The local log subsystem on the Raspberry Pi was monitored to confirm that the internal Syscheck engine immediately flagged the filesystem mutation:

```
sudo tail -f /var/ossec/logs/ossec.log | grep "test_eicar.txt"
```

Step 3: Verification of SIEM Manager & API Threat Enrichment

The core logging pipe on the central Wazuh Manager was audited to verify that the incoming FIM event triggered Rule 554 and successfully automated the outbound API call to the threat intelligence service:

```
sudo tail -f /var/ossec/logs/integrations.log | grep -i virustotal
```

5. Production Evidence & Telemetry Analysis

The verification simulation successfully traversed the entire data pipeline. Security events captured on the main analytical interface demonstrate both automated ingestion and external verification.

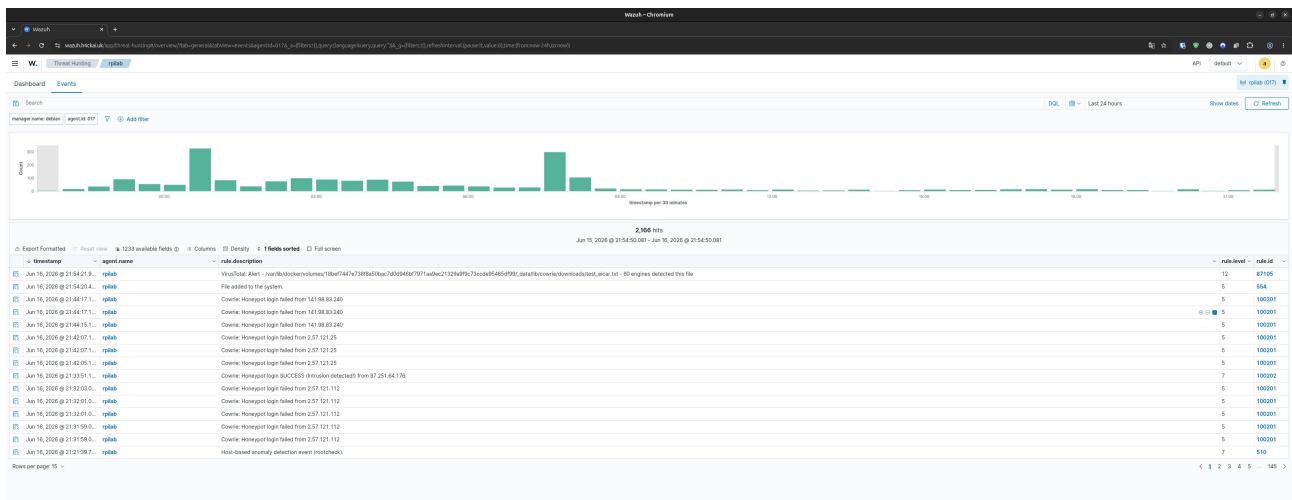


Figure 1: Live Wazuh Threat Hunting dashboard confirming the interception of the payload and the successful firing of the rule pipeline.

